

	SOPORTE DE EJECUCIÓN	
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null) Ciclo: 1

Tabla de Contenido

Definir criterios de completitud del diseño.....	2
Propósito.....	2
Niveles de diseño considerados.....	2
Criterios establecidos en el proyecto.....	2
Estándares de implementación.....	7
Revisión de estándares.....	7
Estándares de nombres, interfaces y mensajes.....	7
Estándares de codificación.....	8
Estándares de tamaño.....	9
Estándares de defectos.....	9
Prevención de defectos.....	10
Estrategia de implementación.....	11
Revisiones – Estrategia Bottom-up.....	11
Reutilización – Prácticas simples de implementación.....	12
Pruebas – Diseñar con Testability.....	13
Revisiones e inspecciones de código.....	16
1) Defectos aleatorios y costo de detectarlos solo con pruebas.....	16
2) Probar “todo” es inviable; inspecciones rinden más.....	16
3) Inspeccionar diseño (no solo código).....	16
4) Flujo recomendado (de diseño detallado a código).....	17
5) Codificación y code review.....	18
6) Inspección de código (1): tiempos y proporciones.....	18
7) Inspección de código (2): umbrales de eficacia.....	19
8) Revisión de calidad del componente y liberación a baseline.....	19
Entregables y evidencias.....	20
Beneficios observados (proyecto completo).....	20

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

Definir criterios de completitud del diseño

Propósito

Antes de iniciar la implementación, se verificó que la fase de diseño estuviera completamente finalizada y aprobada. Esto aseguró insumos claros para construir sin ambigüedades y previno defectos posteriores en integración, rendimiento y experiencia de usuario.

Niveles de diseño considerados

Los criterios se aplicaron en varios niveles, en función de la complejidad del sistema:

- Sistema: arquitectura general React (SPA) + Django (API REST) + Azure SQL (Microsoft SQL server).
- Subsistema: frontend (navegación, estado, componentes compartidos) y backend (apps, servicios, serializadores, repositorios).
- Producto: módulos funcionales: Autenticación y Gestión de usuarios, Dashboard, KPIs, Proyectos, Tareas, Reportes, Configuración.
- Componente: API REST por dominio, modelos de datos, pantallas, componentes UI reutilizables, hooks y utilidades.
- Módulo: clases/servicios de negocio (AuthService, UserService, ProjectService, TaskService, KpiService, ReportService) y sus vistas React correspondientes.

Criterios establecidos en el proyecto

Para cada módulo implementado en esta fase, se verificó que estuvieran completos los siguientes elementos antes de comenzar a programar:

- 1) Arquitectura aprobada

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Separación clara entre frontend, backend y base de datos; definición de puntos de integración y flujo de datos.
- Decisiones de estado de sesión (tokens, renovación, expiración), CORS, y estrategia de manejo de errores transversal.

2) Modelo de datos

- Entidades principales y relaciones: Usuario/Rol/Permiso, Proyecto, Tarea, Registro de Tiempo, KPI, Valor KPI, Configuración de usuario.
- Restricciones de integridad (unicidad, no nulos, FK), índices en campos de búsqueda (usuario, estado, fechas, proyecto) y migraciones numeradas y ordenadas.
- Consideraciones para históricos (auditoría básica de cambios en entidades críticas y bitácora de eventos clave).

3) Contratos de API (Django REST)

- Endpoints y métodos definidos y versionados (/api/v1/...) para:
Auth y Gestión de usuarios (login, register, refresh, roles/permisos, recuperación de contraseña),
Dashboard (resúmenes agregados),
KPIs (CRUD de métricas, configuración/validación de fórmulas, valores, periodos),
Proyectos (CRUD, estados, miembros),
Tareas (CRUD, asignación, registro de tiempo, ciclo de vida),
Reportes (filtros, exportaciones).
- Esquemas JSON de request/response con ejemplos y códigos de estado; mensajes de error estandarizados.
- Reglas transversales: paginación, filtros, ordenamiento, control de permisos por rol y trazabilidad (IDs, timestamps).

4) Diseño de interfaz de usuario

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Wireframes y navegación cerrados para: Login/Registro/Olvidé contraseña, Dashboard, KPIs (editor/visor), Proyectos, Tareas, Reportes, Configuración, Gestión de usuarios.
- Estados de UI especificados (cargando, vacío, error, éxito), validaciones y mensajes consistentes.
- Patrones reutilizables (tabla, formulario, modal, cards, filtros, paginación) y guías de accesibilidad básicas (focus, aria, contraste).

5) Reglas de negocio

- Permisos y roles: administración de usuarios/roles; acciones restringidas por rol (crear proyecto, asignar tarea, ver reportes).
- Tareas – ciclo de vida: ABIERTO → EN_PROGRESO → TERMINADO; restricciones: no completar si no tiene tiempo registrado, no tiene usuario asignado o no pertenece a un proyecto válido.
- KPIs – configuración segura: validador sintáctico y semántico de fórmulas; límites de periodo, fuentes de datos permitidas y manejo de faltantes.
- Proyectos: unicidad por nombre + propietario; cambios de estado sujetos a permisos.
- Reportes: filtros obligatorios para evitar consultas costosas; límites de exportación y paginación.

6) Plan de pruebas de negocio

- Unitarias por clase de servicio (Auth, User, Project, Task, Kpi, Report) con casos de éxito y error.
- Pruebas de integración de endpoints críticos (auth, completar tarea, registrar tiempo, cálculo de KPI, exportar reporte).

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Datos de prueba definidos (usuarios de roles distintos, proyectos de ejemplo, tareas con y sin tiempo, KPIs de distintos tipos).

7) Seguridad mínima

- Política de contraseñas: mínimo 8 caracteres, al menos una mayúscula y al menos un carácter especial.
- Almacenamiento seguro: contraseñas con hash robusto; tokens de autenticación con expiración y mecanismo de refresh.
- Protección de configuración: variables sensibles en .env y fuera del repositorio; claves y cadenas de conexión segregadas por entorno.
- CORS y cabeceras endurecidas en backend; rate limiting y registro de eventos de autenticación básicos.

8) Aprobación formal

- Revisión/inspección de diseño concluida antes de codificar: diagramas, contratos de API, reglas de negocio, UI, plan de pruebas y seguridad.
- Registro de aprobación en el repositorio (PRs de diseño, actas y enlaces a artefactos), sin pendientes críticos abiertos.

Ejemplo aplicado

- Autenticación y Gestión de usuarios:
Modelo de usuario y roles; endpoints de login/register/refresh/recuperación; política de contraseñas aplicada; tokens con expiración; pantallas de acceso y administración de usuarios; pruebas para credenciales inválidas, duplicados y expiración.
- Dashboard:
Definición de agregados (conteo de tareas por estado, proyectos activos, KPIs destacados); API para resúmenes con filtros por usuario/periodo; UI con

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

tarjetas y enlaces contextuales a detalle; prueba de tiempos de respuesta con dataset representativo.

- **KPIs:**
Modelo de KPI y series/valores; validador de fórmulas y periodos; endpoints para CRUD y cálculo/lectura; UI de edición y visualización (gráficas/tablas); reglas para entradas incompletas; pruebas de fórmula válida/ inválida y cálculo por rango.
- **Proyectos:**
CRUD con propietarios y miembros; reglas de estado y permisos; índices por propietario/estado; UI de lista/detalle/edición; pruebas para creación sin nombre, edición sin permiso y archivado.
- **Tareas:**
CRUD + asignación, registro de tiempo y completado con restricciones; transiciones de estado; UI con filtros y acciones; pruebas de negocio (no completar sin tiempo/usuario/proyecto; registro de tiempo negativo).
- **Reportes:**
Endpoints con filtros y paginación; exportación (CSV/XLSX) con límites; UI de filtros/tabla/exportar; pruebas de exportación con volumen moderado y verificación de encabezados/datos.
- **Configuración:**
Preferencias de usuario (tema, idioma, notificaciones); endpoints y persistencia; UI de edición con feedback; pruebas de actualización y validación.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

Estándares de implementación

Revisión de estándares

Se llevó a cabo una revisión pragmática de estándares y se consolidó un marco único para todo el equipo.

- Las reglas de estilo y documentación quedaron formalizadas y publicadas en el repositorio, con ejemplos por módulo.
- Se uniformó la escritura del código y la organización de carpetas (frontend, backend, docs, scripts).
- A partir de la experiencia de los primeros *pull requests*, se ajustaron convenciones y se cerraron como norma estable. Desde entonces, todos los PR deben cumplir con los estándares para ser aprobados.

Estándares de nombres, interfaces y mensajes

Los estándares definidos en diseño están completamente aplicados en la implementación.

- Nombres
 - Componentes de React en PascalCase.
 - Variables y funciones en camelCase.
 - Tablas y campos de BD en snake_case.
- Interfaces
 - Endpoints versionados (/api/v1/...).
 - Recursos en plural (/projects/, /tasks/).

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Métodos HTTP consistentes (GET, POST, PUT/PATCH, DELETE).

- Mensajes

Estructura de error unificada en toda la API:

```
{ "ok": false, "code": "VALIDATION_ERROR", "message": "Campo requerido" }
```

-
- Glosario de nombres
 - Términos como Project, Task, Assignee, Status se usan de forma coherente en FE y BE.
- Control
 - Revisiones de PR verifican nomenclatura, contratos y shape de respuestas. Cualquier desviación se corrige antes del *merge*.

Estándares de codificación

El equipo utiliza y hace cumplir las mismas normas de programación en todo el repositorio.

- Frontend (React)
 - Componentes funcionales con hooks.
 - ESLint y Prettier configurados y ejecutados en *pre-commit*.
 - Servicios HTTP centralizados para todas las llamadas a la API, evitando duplicidad.
- Backend (Django)
 - Cumplimiento estricto de PEP8.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Separación en apps (auth, projects, tasks, KPI) y capas (serializers, services).
- Docstrings y comentarios descriptivos en servicios y serializadores.
- Comentarios y encabezados
 - Cada archivo inicia con encabezado estándar (autor, fecha, propósito).
- Colaboración
 - Los PR incluyen descripción clara, pasos de prueba y evidencias (capturas/colección de endpoints).
 - No se aprueban PR sin pasar lint y pruebas definidas para el alcance del cambio.

Estándares de tamaño

Las métricas de tamaño están operativas para código y documentación.

- LOC (Lines of Code)
 - Las líneas eliminadas se cuentan como comentario para mantener trazabilidad.
- Documentación (páginas)
 - Se mantienen conteos de: SRS, diseño de alto nivel, diseño detallado, pantallas/reportes, base de datos, mensajes/contratos de API, planes y casos de prueba.
- Uso
 - Estas métricas guían la planificación de revisiones (módulos más grandes ⇒ revisión más profunda).
 - Los informes de tamaño se anexan al *baseline* de cada iteración.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

Estándares de defectos

Se adoptó una tipificación simple y efectiva (PSP) y (TSP) para análisis y mejora del proceso.

- Datos (70): integridad y validaciones (BD y formularios).
- Función (80): reglas de negocio (p. ej., no completar tarea sin tiempo).
- Sistema (90): integración FE–BE (p. ej., desajustes en contratos).
- Aplicación
 - Todos los defectos reportados en issues/PR quedan etiquetados con el tipo correspondiente.
 - Los reportes periódicos permiten priorizar acciones de mejora sobre las categorías más recurrentes.

Prevención de defectos

La prevención es parte del proceso operativo y se ejecuta en cada ciclo.

- Educación
 - Guías internas de buenas prácticas en Python, Django y React; sesiones cortas de *knowledge sharing*.
- Comunicación
 - Reuniones cortas para alinear el uso de estándares y resolver dudas de integración.
- Herramientas
 - Migraciones de Django y tareas automatizadas reducen errores mecánicos.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Supervisión
 - Checklists de PR incluyen estilo, seguridad y pruebas mínimas.
- Efectividad
 - Se priorizan defectos frecuentes (p. ej., validaciones faltantes) y se ajusta el proceso para que no reaparezcan (validadores compartidos, componentes form-control reutilizables, etc.).

Estrategia de implementación

Revisiones – Estrategia *Bottom-up*

Para asegurar calidad desde la base, se revisó de abajo hacia arriba:

- Capas y objetos de menor nivel (primero):
 - Modelos y entidades: Usuario/Rol/Permiso, Proyecto, Tarea, Registro de Tiempo, KPI, Valor de KPI, Preferencias de Usuario.
 - Serializadores/DTOs y validadores (Django REST): formatos, tipos, rangos, mensajes de error.
 - Utilidades puras en FE (helpers, hooks sin efectos, form validators).
 - Ejemplos: validación de credenciales en login; formato de fechas; normalización de estados de tarea; verificación de fórmula KPI.
- Integraciones de nivel medio (después):
 - Endpoints por dominio: auth/usuarios, dashboard (resúmenes), KPIs (CRUD/lectura/cálculo), proyectos (CRUD/miembros), tareas (CRUD/asignación/tiempo/ciclo), reportes (filtros/exportaciones),

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

configuración (preferencias).

- Reglas de negocio en servicios (transiciones de estado, permisos por rol, límites de exportación, validación de fórmulas).
- Nivel alto (al final):
 - Vistas y flujos de UI en React: Login/Registro/Olvidé, Dashboard, KPIs (editor/visor), Proyectos, Tareas, Reportes, Configuración, Gestión de usuarios.
 - Navegación protegida, estados (loading/empty/error/success) y experiencia del usuario.
- Resultado: cada capa se validó antes de construir la siguiente, reduciendo defectos de integración y repeticiones de trabajo.

Reutilización – Prácticas simples de implementación

Se aplicaron prácticas para maximizar reuso y consistencia:

- Encabezados estándar en todo archivo fuente (autor, fecha, propósito, módulo).
- Frontend (React):
 - Cliente HTTP centralizado (interceptores, manejo de errores, headers con token).
 - Componentes reutilizables: Form, Input, Select, Modal, Table, Card, KpiChart, FilterBar, Pagination.
 - Hooks compartidos: useAuth, useFetch, useForm, usePagination, useDebounce.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Temas y tokens de diseño (espaciados, tipografía) para uniformidad en Dashboard, KPIs, Proyectos, Tareas, Reportes y Configuración.
- Backend (Django REST):
 - Validadores/serializadores reutilizables (por ejemplo, validador de contraseñas y de fórmulas KPI).
 - Servicios de dominio (AuthService, UserService, ProjectService, TaskService, KpiService, ReportService) para encapsular reglas y facilitar pruebas.
 - Mixins y filtros comunes (paginación, búsqueda, ordenamiento, control por rol).
- Mensajería y contratos uniformes:
 - Estructura estándar de éxito/error en todas las respuestas (ok/code/message/details).
- Resultado: menos duplicación, menor costo de mantenimiento y comportamiento consistente en todos los módulos.

Pruebas – Diseñar con *Testability*

Se diseñó la solución para ser probable y verificable desde el inicio:

- Separación de responsabilidades: lógica de negocio en servicios reutilizables (testables en aislamiento), UI delgada.
- Plan de pruebas por dominio (unitarias + integración de API):
 - Autenticación y Gestión de usuarios
 - *Unitarias*: validación de política de contraseñas (≥ 8 , 1 mayúscula, 1 especial), login con credenciales inválidas,

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

refresh token, alta de usuario duplicado.

- *Integración:* flujo Login → acceso a ruta protegida; alta de usuario → asignación de rol → restricción por rol.
- Dashboard
 - *Unitarias:* agregadores (conteo tareas por estado, proyectos activos, KPIs destacados) con distintos datasets.
 - *Integración:* respuesta $\leq$ umbral local con filtros por usuario/periodo; shape de tarjetas correcto.
- KPIs
 - *Unitarias:* validador sintáctico/semántico de fórmulas, cálculo por periodo, manejo de valores faltantes.
 - *Integración:* CRUD de KPI, lectura de series, cálculo con rangos; errores claros en fórmulas inválidas.
- Proyectos
 - *Unitarias:* crear/editar/eliminar con permisos; unicidad de nombre; cambio de estado.
 - *Integración:* dueño vs. miembro vs. admin; filtros por estado/propietario.
- Tareas
 - *Unitarias:* crear/asignar, registrar tiempo (minutos ≥ 0), completar con precondiciones.
 - *Integración:* no completar sin tiempo/usuario/proyecto; listas filtradas por estado/usuario/proyecto.
- Reportes

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- *Unitarias*: composición de consultas con filtros; formateo de encabezados/footers.
 - *Integración*: paginación, límites de exportación (CSV/XLSX), coherencia de columnas/datos.
- Configuración
 - *Unitarias*: actualización de preferencias (tema/idioma/notificaciones).
 - *Integración*: persistencia por usuario; lectura tras re-login.
- Herramientas de verificación:
 - Colección Postman versionada para todos los endpoints.
 - Datos semilla para pruebas repetibles (usuarios con distintos roles, proyectos y tareas de ejemplo, KPIs de varios tipos).
 - Checks automáticos en PR: linters, formateo, y suites de pruebas mínimas por alcance.
- Front-end (pruebas funcionales ligeras):
 - Validaciones visibles, estados de error/success, transición entre vistas, protección de rutas.
- Resultado: cobertura efectiva de los casos críticos, detección temprana de regresiones y evidencia clara de cumplimiento de criterios de aceptación por módulo.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

Revisiones e inspecciones de código

1) Defectos aleatorios y costo de detectarlos solo con pruebas

En la implementación se asume la presencia de defectos de transcripción (tecleo, nombres mal escritos, orden de parámetros). Para evitarlos temprano:

Cómo se aplica en el proyecto (ya operativo):

- Antes de integrar UI↔API, se ejecuta lint automático y revisión visual rápida en módulos base:
 - BE: modelos, *serializers*, *services* y *permissions* de Django (Auth/Users, Projects, Tasks, KPIs, Reports).
 - FE: validadores de formularios (Login/Register/KPIs/Proyectos/Tareas), *hooks* y contratos de *fetch*.
- Resultado: errores triviales se corrigen antes del E2E, reduciendo tiempos de depuración en sistema.

2) Probar “todo” es inviable; inspecciones rinden más

El espacio de estados (rutas, parámetros, roles, datos) es grande; la prueba exhaustiva no es eficiente.

Cómo se aplica:

- Inspecciones tempranas sobre funciones críticas: login, refresh_token, assign_task, log_time, complete_task, calculate_kpi, export_report.
- Se verifican todas las decisiones y retornos (incluidos bordes y errores) en revisión estática; las pruebas se enfocan en rutas representativas y casos de mayor riesgo.

3) Inspeccionar diseño (no solo código)

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

Los defectos de diseño sofisticados no aparecen fácilmente en el código.
Cómo se aplica:

- Cada módulo tuvo diseño completo y aprobado: modelo de datos, contratos API, reglas de negocio, flujos de UI, y política de contraseñas (≥ 8 , 1 mayúscula, 1 especial).
- Ejemplos aclarados en diseño:
 - Tareas: quién puede completar, precondiciones (tiempo registrado, usuario asignado, proyecto válido), estados ABIERTO→EN_PROGRESO→TERMINADO.
 - KPIs: validador de fórmulas y fuentes permitidas.
 - Reportes: límites de exportación y filtros obligatorios.

4) Flujo recomendado (de diseño detallado a código)

Secuencia aplicada por dominio (Auth/Users, Dashboard, KPIs, Projects, Tasks, Reports, Config):

1. Diseño detallado (datos, endpoints, UI, reglas, seguridad).
2. Revisión personal del diseño por el autor.
3. Utilidades de prueba preparadas (Postman, datos semilla por rol/volumen).
4. Inspección de diseño por un revisor (mínimo 1):
 - Para funciones con bucles/ramas, completar tabla de ejecución (entradas → decisiones → salidas).
5. Codificación del módulo.
Ejemplo – Tareas: transición abierta→en progreso→terminado, regla “no completar sin tiempo/usuario/proyecto”, tabla de entradas/salidas y luego codificación.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

5) Codificación y *code review*

El autor revisa su historial de defectos y fija un tiempo objetivo de auto-revisión antes del PR.

Criterios en PR (aplicación transversal):

- Consistencia de nombres (componentes, funciones, DTOs/serializers).
- Inicializaciones y valores por defecto.
- Asignaciones/validaciones (p. ej., minutos ≥ 0 en log_time, formato de email/contraseña).
- Firmas y retornos: códigos HTTP, forma JSON (ok/code/message/details).
- Seguridad y UX: no exponer secretos, CORS controlado, mensajes claros, estados loading/error/empty/success.
- Evidencia: capturas, *curl* o Postman de endpoints tocados.

6) Inspección de código (1): tiempos y proporciones

Se comparan los tiempos con el plan de calidad:

- Diseño $\geq$ Codificación del módulo.
- Revisión de diseño $\geq 50\%$ del tiempo de diseño.
- Revisión de código $\geq 50\%$ del tiempo de código (ideal $> 75\%$).
Cómo se registra: en cada PR se declara esfuerzo (diseño/código/revisión). Si la revisión cae por debajo del umbral, se solicita revisión adicional antes del *merge*.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

7) Inspección de código (2): umbrales de eficacia

La inspección se considera efectiva cuando:

- En el *code review* se encuentran $\geq 2\times$ defectos que los detectados por *lint/ejecución local*.
- Se detectan > 1 defectos/h (cuando hay defectos).
- La velocidad de revisión es < 200 LOC/h (profundidad $>$ velocidad).
- Si persisten dudas de calidad, se reinspecciona el componente antes de corregir los defectos conocidos, para evitar “parches” superficiales..
Aplicado: especial foco en Auth y Tasks: *serializers/services* y la concordancia exacta de DTOs FE $\leftrightarrow$ BE; en KPIs, el validador y cálculos por periodo; en Reportes, paginación y exportación.

8) Revisión de calidad del componente y liberación a *baseline*

Antes de liberar un componente a la línea base:

- Cada segmento lógico relevante se analiza al menos una vez con tabla de ejecución.
- El líder de soporte confirma listo para baseline (cumple diseño, reglas y criterios de salida de la fase).
- Si una reinspección insume horas extra, se realiza (ahorra días en integración y pruebas de sistema).
- Se aprueba el merge y se etiqueta la versión del componente.

Tablas de ejecución (resumen por dominio):

- Auth/Users: login(email, password) $\rightarrow$ inexistente/ inválido/ válido; register() $\rightarrow$ email duplicado; refresh_token() $\rightarrow$ expirado vs válido.

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Projects: create_project(name, owner) → vacío/owner inexistente/éxito; update_project() permisos.
- Tasks: assign_task(task, user) → usuario inválido; log_time(task, minutes) → <0 / válido; complete_task(task) → sin tiempo/sin usuario/sin proyecto/éxito.
- KPIs: validate_formula(expr) → sintaxis/semántica; calculate(kpi, range) → sin datos/con datos; bordes de periodo.
- Reports: export(filter) → sin filtros requeridos / límite de filas / formato; paginación correcta.
- Config: update_preferences(user, prefs) → claves inválidas/éxito; lectura tras re-login.

Entregables y evidencias

- Comentarios estándar en archivos y PRs con: descripción del cambio, casos cubiertos y tabla de ejecución en funciones críticas.
- Registro de tiempos (diseño/código/revisión) en la descripción del PR o tablero de control.
- Aprobación de calidad del componente (estado “OK para baseline”) y tag de versión.

Beneficios observados (proyecto completo)

- Reducción de errores triviales (nombres/firmas/validaciones).
- Integración FE–BE más fluida (contratos estables, menos “rompimientos”).

	SOPORTE DE EJECUCIÓN		
Universidad Piloto de Colombia	PROYECTO: DashKPIs	Grupo: Print(Null)	
		Ciclo:	1

- Flujos clave listos con menos retrabajo:
 - Auth/Users (login/register/refresh, política de contraseñas ≥ 8 con mayúscula y caracter especial).
 - Dashboard (resúmenes consistentes por usuario/periodo).
 - KPIs (configuración validada, cálculos confiables).
 - Proyectos (CRUD y permisos).
 - Tareas (CRUD, asignación, tiempo, completar con precondiciones).
 - Reportes (filtros, paginación y exportación estable).
 - Configuración (preferencias por usuario persistentes).

CONTROL DE CAMBIOS		
Fecha	Descripción	Autor(es)
30/09/2025	Se realiza el soporte de ejecución	Leidy Giraldo Kerlon Gutierrez